

Assignment 1 - PS07: Emergency Route Planning using Greedy Best First Search

Group Id: G087

1. Problem Formulation

The disaster management control center needs a problem-solving agent that finds an emergency route from a start location to a goal location during flooding in Chennai. The city map is represented as a graph where each node is a location and each edge is a reachable road segment. The agent uses Greedy Best First Search, where $f(n) = h(n)$, so the next expanded node is always the frontier node with the smallest estimated distance to the goal.

Initial State: the user-provided start node.

Goal Test: current node equals the goal node.

Actions: move from the current location to any connected neighboring location.

Path Cost: sum of edge costs along the returned route. If no edge cost is supplied in the input file, cost 1 is used.

2. PEAS Description

Performance Measure: reach the affected area quickly, produce a valid path, minimize heuristic distance at every choice, report the visited order, path, cost, measured runtime, and measured memory.

Environment: Chennai road network during heavy rain, including possible flooded or blocked roads represented by missing or high-cost edges.

Actuators: route selection decisions, expansion of next city node, and output of route guidance to rescue teams.

Sensors: input graph, heuristic table, start node, goal node, and edge-cost information.

3. Environmental Characteristics

The environment is partially observable because the program only knows the graph and heuristic values provided in the input. It is deterministic for the submitted implementation because each action leads to the listed neighbor with a known cost. It is sequential because each route decision affects the following state. It is static during one search run because the graph is not changed while the algorithm executes. It is discrete because locations and roads are represented as graph nodes and edges. It is single-agent from the search perspective because only one rescue route is planned by this agent.

4. Heuristic Function

The heuristic $h(n)$ estimates how close a node n is to the goal. In this assignment's sample cases, lower heuristic values indicate locations that appear closer to the affected area.

GBFS ignores accumulated path cost while choosing the next node; it expands the frontier node with the smallest $h(n)$. This makes the search fast and goal-directed, but it does not guarantee the minimum-cost path for all weighted graphs.

5. Data Structures and Algorithm

The graph is stored as an adjacency list: node $\rightarrow$ list of (neighbor, cost). Heuristic values are stored in a dictionary for $O(1)$ lookup. The frontier is a bounded min-priority queue implemented using heapq. The queue explicitly raises an error if insertion is attempted when full or deletion is attempted when empty, satisfying the data-structure error-handling requirement. A visited set prevents repeated expansion of the same node. Each priority queue entry stores heuristic value, insertion order, current node, path so far, and accumulated cost.

Algorithm steps:

1. Read graph, heuristics, start node, and goal node from the input file.
2. Insert the start node in the priority queue with priority $h(\text{start})$.
3. Repeatedly remove the node with the lowest heuristic value.
4. If the node is the goal, print expansion order, path, cost, measured time, and measured memory.
5. Otherwise, insert all unvisited neighbors with priority $h(\text{neighbor})$.
6. If the frontier becomes empty, report that no route exists.

6. Complexity Reporting

The program prints measured runtime using `time.perf_counter()` and measured peak memory using `tracemalloc` for the actual input run, as required by the assignment instruction not to calculate these theoretically in the implementation. For design understanding, if V is the number of locations and E is the number of roads, the priority queue operations make the expected algorithmic time $O(E \log E)$ in the worst case and the auxiliary space $O(E)$ for the frontier plus visited/path information.

7. Alternate Modeling Approach

An alternate model is to represent the problem as a weighted shortest-path search using A^* with $f(n) = g(n) + h(n)$, where $g(n)$ is the cost from the start and $h(n)$ is the estimated

remaining cost. A* generally has higher bookkeeping cost than GBFS because it must track and compare accumulated path costs, but it can find an optimal path when the heuristic is admissible and consistent. GBFS is simpler and often faster for emergency guidance when a quick feasible route is more important than guaranteed optimality, but it may choose a longer route if the heuristic is misleading.

Comparison with Other Search Algorithms:

While implementing the routing agent, we also examined other search strategies that could be used for the same problem. Greedy Best First Search was selected because it quickly moves toward the goal using heuristic information. However, other search algorithms have different strengths and limitations.

Algorithm	Uses Heuristic	Uses Path Cost	Optimal Path Guaranteed
Greedy Best First Search (GBFS)	Yes	No	No
A* Search	Yes	Yes	Yes (with admissible heuristic)
Breadth First Search (BFS)	No	No	Yes for unweighted graphs
Depth First Search (DFS)	No	No	No

For emergency response situations, GBFS can often produce a feasible route faster than exhaustive search methods because it focuses on locations that appear closer to the destination. However, the quality of the selected route depends on the accuracy of the heuristic values.

8. Sample Result

Search Progression for Test Case 1

Search Progression for Test Case 1

The execution of Greedy Best First Search for Test Case 1 is summarized below.

Step 1:

The search begins at node A.

The neighboring nodes available for expansion are B and C.

Their heuristic values are $h(B)=8$ and $h(C)=5$.

Since node C has the smaller heuristic value, it is selected for expansion.

Current Expansion Order:

$A \rightarrow C$

Step 2:

After expanding node C, the frontier contains nodes B and F.

Their heuristic values are $h(B)=8$ and $h(F)=2$.

Node F is selected because it has the smallest heuristic value among the available frontier nodes.

Current Expansion Order:

$A \rightarrow C \rightarrow F$

Step 3:

After expanding node F, node G becomes available in the frontier.
Since $h(G)=0$, it is selected immediately.

Final Expansion Order:

$A \rightarrow C \rightarrow F \rightarrow G$

The goal node is reached and the search terminates.

Final Path:

$A \rightarrow C \rightarrow F \rightarrow G$

This example demonstrates how Greedy Best First Search always chooses the frontier node with the lowest heuristic value. The algorithm quickly reaches the destination by continuously selecting the node that appears closest to the goal according to the heuristic information.

In summary :

For Test Case 1, start A and goal G, the program expands:

$A \rightarrow C \rightarrow F \rightarrow G$

The returned path is:

$A \rightarrow C \rightarrow F \rightarrow G$

The total cost is 3 when every road segment has unit cost.

9.Observations

During testing, it was observed that Greedy Best First Search reaches the goal node with very few node expansions when suitable heuristic values are available. This makes the algorithm attractive for emergency route planning where a quick decision is often more important than finding the absolute best route.

Another observation is that the algorithm relies heavily on the heuristic information provided. If the heuristic values do not accurately represent closeness to the goal, the selected route may not be the lowest-cost route. Therefore, the effectiveness of GBFS is strongly influenced by the quality of the heuristic function.